

Rapport TP : Communication RS485 et Modbus avec ESP32

L'objectif de ce TP est de réaliser une communication entre deux ESP32 grâce au protocole RS485.

Trois cas ont été étudiés : une communication simple, un échange de données (température) et une communication structurée avec le protocole Modbus.

1. Cas 1 : Communication simple RS485 :

Objectif : Envoyer un message texte entre deux ESP32.

- Émission : L'ESP32 en mode émetteur envoie un message via Serial2 après activation du mode émission (RS485_EN = HIGH.)

```
// CAS 1 : EMISSION RS485 SIMPLE
#define RXD2 16
#define TXD2 17
#define RS485_EN 19

void setup() {
  Serial.begin(115200);
  Serial2.begin(9600, SERIAL_8N1, RXD2, TXD2);

  pinMode(RS485_EN, OUTPUT);
  digitalWrite(RS485_EN, LOW);

  Serial.println("Emetteur RS485 prêt");
}

void loop() {
  digitalWrite(RS485_EN, HIGH); // mode émission
  delay(2);

  Serial2.println("Bonjour depuis ESP32");

  Serial2.flush();
  delay(2);
  digitalWrite(RS485_EN, LOW); // retour réception

  Serial.println("Message envoyé");
  delay(2000);
}
```

- Réception : L'ESP32 récepteur lit le message via Serial2 en mode réception (RS485_EN = LOW).

```
// CAS 1 : RECEPTION RS485 SIMPLE
#define RXD2 16
#define TXD2 17
#define RS485_EN 19

void setup() {
  Serial.begin(115200);
  Serial2.begin(9600, SERIAL_8N1, RXD2, TXD2);

  pinMode(RS485_EN, OUTPUT);
  digitalWrite(RS485_EN, LOW); // mode réception

  Serial.println("Récepteur prêt");
}

void loop() {
  if (Serial2.available()) {
    String msg = Serial2.readStringUntil('\n');
    msg.trim();

    Serial.print("Reçu : ");
    Serial.println(msg);
  }
}
```

➤ le message est transmis correctement et s'affiche sur le moniteur série.

2. Cas 2 : Transmission de température (sans Modbus)

Objectif : Transmettre une donnée numérique (température).

- Emission : lecture d'une température (par capteur DHT) et transmission par RS485.

```
// CAS 2 : EMISSION TEMPERATURE SANS MODBUS
#define RXD2 16
#define TXD2 17
#define RS485_EN 19

void setup() {
  Serial.begin(115200);
  Serial2.begin(9600, SERIAL_8N1, RXD2, TXD2);

  pinMode(RS485_EN, OUTPUT);
  digitalWrite(RS485_EN, LOW);

  Serial.println("Emetteur température prêt");
}

void loop() {
  float temperature = 25.5;

  digitalWrite(RS485_EN, HIGH);
  delay(2);

  Serial2.println(temperature);

  Serial2.flush();
  delay(2);
  digitalWrite(RS485_EN, LOW);

  Serial.print("Température envoyée : ");
  Serial.println(temperature);

  delay(2000);
}
```

o Réception : on lit et affiche la valeur reçue.

```
// CAS 2 : RECEPTION TEMPERATURE SANS MODBUS
#define RXD2 16
#define TXD2 17
#define RS485_EN 19

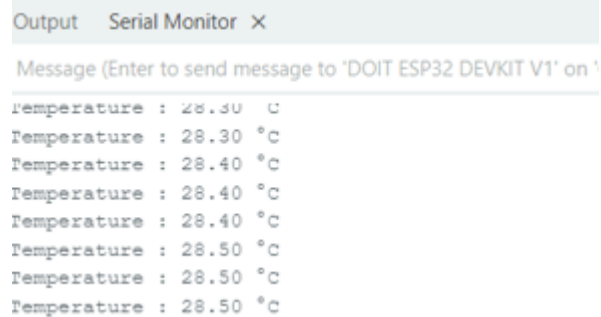
void setup() {
  Serial.begin(115200);
  Serial2.begin(9600, SERIAL_8N1, RXD2, TXD2);

  pinMode(RS485_EN, OUTPUT);
  digitalWrite(RS485_EN, LOW);

  Serial.println("Receveur prêt - attente température");
}

void loop() {
  if (Serial2.available()) {
    String temperature = Serial2.readStringUntil('\n');
    temperature.trim();

    Serial.print("Température reçue : ");
    Serial.println(temperature);
  }
}
```



The screenshot shows the 'Serial Monitor' window in the Arduino IDE. The title bar reads 'Output Serial Monitor X'. Below the title bar, there is a text input field with the placeholder 'Message (Enter to send message to 'DOIT ESP32 DEVKIT V1' on ' and a 'Send' button. The main area of the window displays a series of temperature readings received from the device. The readings are: 'temperature : 28.30 C', 'Temperature : 28.30 °C', 'Temperature : 28.40 °C', 'Temperature : 28.40 °C', 'Temperature : 28.40 °C', 'Temperature : 28.50 °C', 'Temperature : 28.50 °C', and 'Temperature : 28.50 °C'.

```
Output Serial Monitor X
Message (Enter to send message to 'DOIT ESP32 DEVKIT V1' on '
temperature : 28.30 C
Temperature : 28.30 °C
Temperature : 28.40 °C
Temperature : 28.40 °C
Temperature : 28.40 °C
Temperature : 28.50 °C
Temperature : 28.50 °C
Temperature : 28.50 °C
```

➤ **Résultat : La température est bien transmise et affichée.**

3. Cas 3 : Communication Modbus avec Holding Register

Dans ce cas, le capteur DHT22 est connecté à l'ESP32 esclave.

Il mesure la température et la stocke dans un holding register.

Le maiter lit cette valeur via Modbus RS485.

- Code Maiter :

```
1  #include <ModbusRTU.h>
2
3  #define RXD2 16
4  #define TXD2 17
5  #define RS485_EN 4
6
7  ModbusRTU mb;
8  typedef __uint16_t uint16_t
9  uint16_t temp ;
10
11 void setup() {
12     Serial.begin(115200);
13     Serial2.begin(9600, SERIAL_8N1, RXD2, TXD2);
14     pinMode(RS485_EN, OUTPUT);
15     mb.begin(&Serial2, RS485_EN);
16     mb.master();
17 }
18
19 void loop() {
20
21     if (!mb.slave()) {
22         mb.readHreg(1, 0, &temp, 1); // température
23     }
24
25     mb.task();
26     Serial.print("Temperature : ");
27     Serial.print(temp / 10.0);
28     Serial.println(" °C");
29     delay(1000);
30 }
```

Output Serial Monitor X

Message (Enter to send message to 'DOIT ESP32 DEVKIT V1' on 'COM5')

```
Temperature : 28.30 °C
Temperature : 28.30 °C
Temperature : 28.40 °C
Temperature : 28.40 °C
Temperature : 28.40 °C
Temperature : 28.50 °C
Temperature : 28.50 °C
Temperature : 28.50 °C
```

- **Code Esclave :**

```
1  #include <ModbusRTU.h>
2  #include <DHT.h>
3
4  #define RXD2 16
5  #define TXD2 17
6  #define RS485_EN 4
7
8  #define DHTPIN 21
9  #define DHTTYPE DHT22
10
11 DHT dht(DHTPIN, DHTTYPE);
12 ModbusRTU mb;
13
14 void setup() {
15     Serial.begin(115200);
16     Serial2.begin(9600, SERIAL_8N1, RXD2, TXD2);
17
18     pinMode(RS485_EN, OUTPUT);
19
20     dht.begin();
21
22     mb.begin(&Serial2, RS485_EN);
23     mb.slave(1);
24     mb.addHreg(0); // température
25 }
26
27 void loop() {
28
29     float temp = dht.readTemperature();
30     if (!isnan(temp) ) {
31         mb.Hreg(0, temp * 10);
32     }
33
34     mb.task();
35 }
```

➤ Résultat : Le Master reçoit la température via Modbus.

- **Conclusion**

La communication RS485 permet un échange fiable entre équipements. L'utilisation de Modbus permet d'organiser les données sous forme de registres. Le Master contrôle la communication tandis que le Slave fournit les données.

